ಆವರ್ತಕ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ಗಳು

RNN, LSTM ಮತ್ತು GRU — “ನೆನಪು” ಇರುವ ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Recurrent Neural Networks — RNNs” ಉಪನ್ಯಾಸ
ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ಅನುಕ್ರಮ ದತ್ತಾಂಶಕ್ಕೆ “ನೆನಪು” ಏಕೆ ಬೇಕು?

೨. ಆವರ್ತಕ ನ್ಯೂರಾನ್: ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿದಾಗ (ಹಂತ-ಹಂತದ ಲೆಕ್ಕ)

೩. ಆವರ್ತಕ ಪದರ, ಮ್ಯಾಟ್ರಿಕ್ಸ್ ರೂಪ ಮತ್ತು ಮೆಮೊರಿ ಸೆಲ್

೪. ಇನ್‌ಪುಟ್-ಔಟ್‌ಪುಟ್ ಅನುಕ್ರಮಗಳ ನಾಲ್ಕು ವಿಧಗಳು

೫. RNN ತರಬೇತಿ (BPTT) ಮತ್ತು Keras SimpleRNN

೬. ಸರಳ RNN ನ ದೌರ್ಬಲ್ಯ: ಮಾಯವಾಗುವ / ಸ್ಫೋಟಿಸುವ ಗ್ರೇಡಿಯಂಟ್

೭. LSTM — ಗೇಟ್‌ಗಳ ಮೂಲಕ ದೀರ್ಘ ನೆನಪು

೮. GRU — ಸರಳೀಕೃತ ಗೇಟ್ ಕೋಶ ಮತ್ತು LSTM ಹೋಲಿಕೆ

೯. IMDB ವಿಮರ್ಶೆಗಳ ಸೆಂಟಿಮೆಂಟ್: LSTM ಮಾದರಿ (ಕೋಡ್ ಮತ್ತು ವಿಶ್ಲೇಷಣೆ)

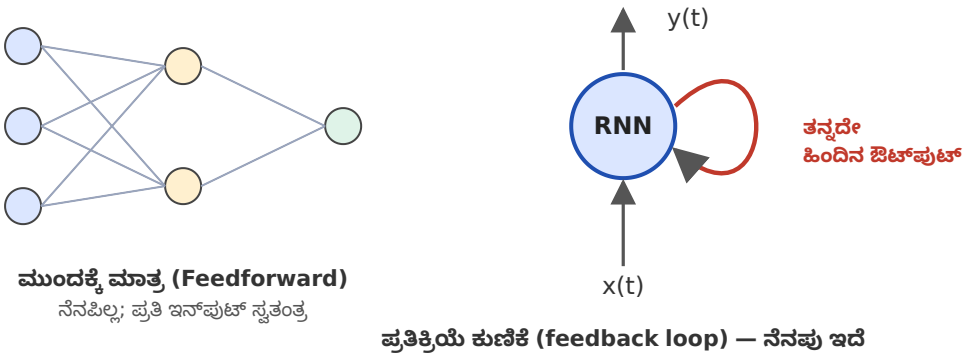
೧೦. GRU ಮಾದರಿ ಮತ್ತು ತರಬೇತಿ ಸಮಯ ಅಳೆಯುವುದು

೧೧. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ಅನುಕ್ರಮ ದತ್ತಾಂಶಕ್ಕೆ “ನೆನಪು” ಏಕೆ ಬೇಕು?

ಇದುವರೆಗೆ ನೋಡಿದ ನೆಟ್‌ವರ್ಕ್‌ಗಳು: **Feedforward** (ಸಂಪೂರ್ಣ ಸಂಪರ್ಕ) ಮತ್ತು **CNN** (ಚಿತ್ರಗಳಿಗೆ). ಇವೆರಡರಲ್ಲಿ ಪ್ರತಿ ಇನ್‌ಪುಟ್ ಇನ್ನೊಂದರಿಂದ ಸ್ವತಂತ್ರ — ನೆಟ್‌ವರ್ಕ್‌ಗೆ “ಈಗ ತಾನೆ ಏನು ನೋಡಿದೆ” ಎಂಬ ನೆನಪಿಲ್ಲ. ಆದರೆ ಪಠ್ಯ, ಧ್ವನಿ, ಷೇರು ಬೆಲೆ, ಹವಾಮಾನ ಮುಂತಾದ **ಅನುಕ್ರಮ (sequence)** ದತ್ತಾಂಶದಲ್ಲಿ **ಕ್ರಮ ಮತ್ತು ಹಿಂದಿನ ಸಂದರ್ಭ** ಅರ್ಥ ನೀಡುತ್ತದೆ.

> **ಉದಾಹರಣೆ: ಕ್ರಮ ಏಕೆ ಮುಖ್ಯ?**
> “ಈ ಚಿತ್ರ **ಚೆನ್ನಾಗಿಲ್ಲ**” ಮತ್ತು “ಈ ಚಿತ್ರ ಚೆನ್ನಾಗಿದೆ, **ಆದರೆ** ಕೊನೆ **ಚೆನ್ನಾಗಿಲ್ಲ**” — ಪದಗಳ ಕ್ರಮ ಮತ್ತು ಹಿಂದಿನ ಪದಗಳ ಸಂದರ್ಭ ಗೊತ್ತಿದ್ದರೆ ಮಾತ್ರ ಸರಿಯಾದ ಭಾವನೆ ತಿಳಿಯುತ್ತದೆ. ಒಂದು ಕಥೆ ಓದುವಾಗ ನೀವು ಪ್ರತಿ ವಾಕ್ಯವನ್ನು ಹಿಂದಿನ ವಾಕ್ಯಗಳ ನೆನಪಿನೊಂದಿಗೆ ಓದುತ್ತೀರಿ; RNN ಕೂಡ ಹಾಗೆಯೇ.



ಚಿತ್ರ ೧: ಎಡ — Feedforward (ನೆನಪಿಲ್ಲ). ಬಲ — Recurrent (ಔಟ್‌ಪುಟ್ ಮತ್ತೆ ಒಳಗೆ ಬರುತ್ತದೆ)

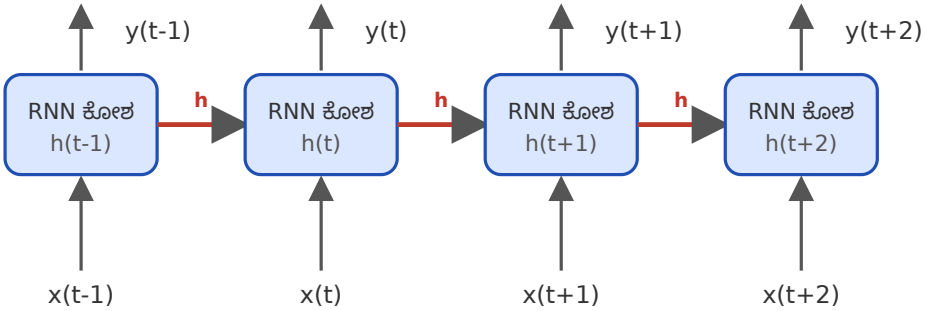
RNN ನ ಲಕ್ಷಣಗಳು

- ಬೇರೆ ಬೇರೆ **ಉದ್ದದ** ಅನುಕ್ರಮಗಳನ್ನು ಸಂಸ್ಕರಿಸಬಲ್ಲದು.
- ಹಿಂದಿನ ಹೆಜ್ಜೆಗಳ ಮಾಹಿತಿಯನ್ನು **ಸಂಗ್ರಹಿಸುತ್ತದೆ**.
- ಕಾಲ-ಸರಣಿ (time series), ಪಠ್ಯ, ಧ್ವನಿಗೆ ಉಪಯುಕ್ತ.
- ಮೊದಲ ಮಾದರಿಗಳು: **Hopfield (1982)**, **Elman (1990, “Finding Structure in Time”)**. ನಂತರ LSTM (Hochreiter ಮತ್ತು Schmidhuber, 1997), GRU (Cho et al., 2014).

೨. ಆವರ್ತಕ ನ್ಯೂರಾನ್: ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿದಾಗ

ಪ್ರತಿ **ಕಾಲ ಹೆಜ್ಜೆ t** (ಇದಕ್ಕೆ “frame” ಎಂದೂ ಹೆಸರು) ದಲ್ಲಿ ಆವರ್ತಕ ನ್ಯೂರಾನ್ ಎರಡು ವಿಷಯಗಳನ್ನು ಪಡೆಯುತ್ತದೆ: ಈಗಿನ ಇನ್‌ಪುಟ್ **x(t)** ಮತ್ತು ತನ್ನದೇ ಹಿಂದಿನ ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್ **y(t−1)**. ಇದರಿಂದ ಒಂದೇ ನ್ಯೂರಾನ್ ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿದರೆ (unroll) ಒಂದೇ ತೂಕಗಳನ್ನು ಬಳಸುವ, ಒಂದರ ಹಿಂದೆ ಒಂದು ಜೋಡಿಸಿದ ಕೋಶಗಳ ಸರಪಳಿಯಂತೆ ಕಾಣುತ್ತದೆ.

ಒಂದೇ ಕೋಶ ಕಾಲ-ಹೆಜ್ಜೆಗಳ ಮೇಲೆ ಮರುಬಳಕೆ (unrolled in time)



ಕೆಂಪು ಬಾಣ = ಹಿಂದಿನ ಹೆಜ್ಜೆಯ “ನೆನಪು” ಮುಂದಿನ ಹೆಜ್ಜೆಗೆ ಹೋಗುತ್ತದೆ

ಚಿತ್ರ ೨: ಒಂದೇ ಕೋಶ (ಒಂದೇ ತೂಕಗಳು) ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ಮರುಬಳಕೆ

ಮುಖ್ಯ ಸೂತ್ರ

```
y(t) = φ( wxᵀ · x(t) + wyᵀ · y(t−1) + b )

φ = ಸಕ್ರಿಯಕರಣ (ಸಾಮಾನ್ಯವಾಗಿ tanh; ReLU ಕೂಡ ಬಳಸುತ್ತಾರೆ)
wx = ಇನ್‌ಪುಟ್‌ಗೆ ತೂಕ,   wy = ಹಿಂದಿನ ಔಟ್‌ಪುಟ್‌ಗೆ ತೂಕ,   b = bias
ಮೊದಲ ಹೆಜ್ಜೆಯಲ್ಲಿ (t = 0): ಆರಂಭಿಕ ಸ್ಥಿತಿ  y(0) = 0
```

ಅಂದರೆ ಪ್ರತಿ ಆವರ್ತಕ ನ್ಯೂರಾನ್‌ಗೆ **ಎರಡು ತೂಕ ಗುಂಪುಗಳು**: ಒಂದು ಈಗಿನ ಇನ್‌ಪುಟ್‌ಗೆ (wx), ಇನ್ನೊಂದು ಹಿಂದಿನ ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್‌ಗೆ (wy).

ಹಂತ-ಹಂತದ ಲೆಕ್ಕ (ನನ್ನ ಸರಳ ಉದಾಹರಣೆ): ಒಂದು ನ್ಯೂರಾನ್, ಮೂರು ಹೆಜ್ಜೆ

ತೂಕಗಳು wx = 0.5, wy = 0.8, b = 0, φ = tanh. ಇನ್‌ಪುಟ್ ಅನುಕ್ರಮ x = [1, 0, 1]. ಆರಂಭ y(0) = 0.

ಹೆಜ್ಜೆ	ಇನ್‌ಪುಟ್ x	ಲೆಕ್ಕ	ಔಟ್‌ಪುಟ್ y
1	1	tanh(0.5×1 + 0.8×0) = tanh(0.5)	**0.4621**
2	0	tanh(0.5×0 + 0.8×0.4621) = tanh(0.3697)	**0.3537**
3	1	tanh(0.5×1 + 0.8×0.3537) = tanh(0.7830)	**0.6544**

ಗಮನಿಸಿ: ಹೆಜ್ಜೆ 2 ರಲ್ಲಿ ಇನ್‌ಪುಟ್ 0 ಆದರೂ ಔಟ್‌ಪುಟ್ 0.3537 (ಸೊನ್ನೆಯಲ್ಲ) — ಹಿಂದಿನ ಹೆಜ್ಜೆಯ "ನೆನಪು" ಉಳಿದಿದೆ. ಹೆಜ್ಜೆ 3 ರ ಫಲಿತಾಂಶ ಹೆಜ್ಜೆ 1 ಮತ್ತು 2 ರ ಇತಿಹಾಸವನ್ನೂ ಒಳಗೊಂಡಿದೆ. ಅಂದರೆ ಅದೇ ಇನ್‌ಪುಟ್ 1 ಗೆ ಹೆಜ್ಜೆ 1 ರಲ್ಲಿ 0.4621 ಮತ್ತು ಹೆಜ್ಜೆ 3 ರಲ್ಲಿ 0.6544 — **ಸಂದರ್ಭದ ಮೇಲೆ ಅವಲಂಬನೆ**.

೩. ಆವರ್ತಕ ಪದರ, ಮ್ಯಾಟ್ರಿಕ್ಸ್ ರೂಪ ಮತ್ತು ಮೆಮೊರಿ ಸೆಲ್

ಒಂದೇ ನ್ಯೂರಾನ್ ಬದಲು ಒಂದು **ಪದರದಲ್ಲಿ n_neurons ನ್ಯೂರಾನ್‌ಗಳು**. ಆಗ ತೂಕ ಸದಿಶಗಳು ಎರಡು ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ಗಳಾಗುತ್ತವೆ: **Wx** ಮತ್ತು **Wy**. ಮಿನಿ-ಬ್ಯಾಚ್ (m ಉದಾಹರಣೆಗಳು) ಗೆ:

```
Y(t) = φ( X(t)·Wx + Y(t−1)·Wy + b ) = φ( [X(t)  Y(t−1)] · W + b ),   W = [Wx ; Wy] (ಲಂಬವಾಗಿ ಜೋಡಣೆ)
```

ಚಿಹ್ನೆ	ಆಕಾರ	ಅರ್ಥ
Y(t)	m × n_neurons	ಪದರದ ಔಟ್‌ಪುಟ್ (ಇಡೀ mini-batch)
X(t)	m × n_inputs	ಇನ್‌ಪುಟ್ (n_inputs = ಇನ್‌ಪುಟ್ ಲಕ್ಷಣಗಳ ಸಂಖ್ಯೆ)
Wx	n_inputs × n_neurons	ಈಗಿನ ಇನ್‌ಪುಟ್‌ಗೆ ತೂಕ
Wy	n_neurons × n_neurons	ಹಿಂದಿನ ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್‌ಗೆ ತೂಕ
b	n_neurons	bias ಸದಿಶ
W (ಜೋಡಿಸಿದ)	(n_inputs + n_neurons) × n_neurons	Wx ಮತ್ತು Wy ಲಂಬವಾಗಿ ಜೋಡಣೆ; [X Y] ಅಡ್ಡವಾಗಿ ಜೋಡಣೆ

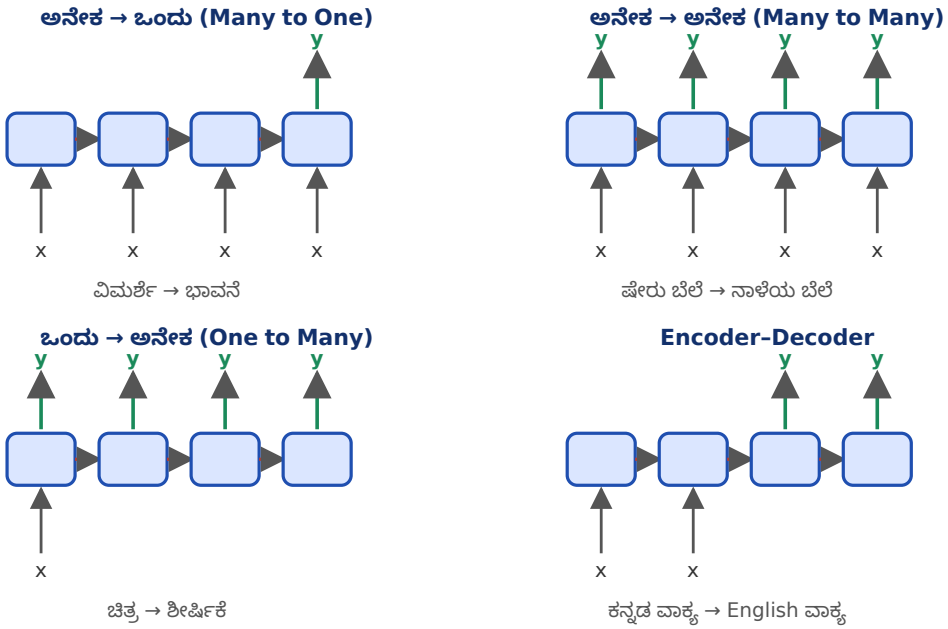
> **ಲೆಕ್ಕ: ಆಕಾರ ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ನನ್ನ ಉದಾಹರಣೆ)**
>
> ಬ್ಯಾಚ್ m = 32, n_inputs = 10, n_neurons = 5:
>
> ```
> X(t): 32×10   Wx: 10×5   Y(t−1): 32×5   Wy: 5×5   b: 5
> X·Wx → 32×5;  Y(t−1)·Wy → 32×5;  ಕೂಡಿ + b → Y(t): 32×5
> ಪ್ಯಾರಾಮೀಟರ್ = n_inputs·n_neurons + n_neurons² + n_neurons = 50 + 25 + 5 = 80
> ```
>
> ಸಾಮಾನ್ಯ ಸೂತ್ರ: **units × (n_inputs + units) + units**. ಅನುಕ್ರಮ ಎಷ್ಟೇ ಉದ್ದವಿದ್ದರೂ ಪ್ಯಾರಾಮೀಟರ್ ಸಂಖ್ಯೆ ಬದಲಾಗುವುದಿಲ್ಲ (ಒಂದೇ ತೂಕಗಳು ಎಲ್ಲ ಹೆಜ್ಜೆಗಳಲ್ಲಿ ಮರುಬಳಕೆ — CNN ನ weight sharing ನಂತೆ, ಆದರೆ ಕಾಲದ ಮೇಲೆ).

ಮೆಮೊರಿ ಸೆಲ್ (Memory Cell)

- ಒಂದು ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್ ಹಿಂದಿನ ಹೆಜ್ಜೆಗಳ ಇನ್‌ಪುಟ್‌ಗಳ ಮೇಲೆ ಅವಲಂಬಿತ → ಒಂದು ರೀತಿಯ ನೆನಪು.
- ಹೆಜ್ಜೆಗಳ ನಡುವೆ ಸ್ಥಿತಿಯನ್ನು ಉಳಿಸುವ ನೆಟ್‌ವರ್ಕ್ ಭಾಗಕ್ಕೆ **ಮೆಮೊರಿ ಸೆಲ್ (cell)** ಎನ್ನುತ್ತಾರೆ.
- ಸೆಲ್‌ನ ಸ್ಥಿತಿ **h(t)** (h = hidden): `h(t) = f( h(t−1), x(t) )`. ಸೆಲ್‌ನ ಔಟ್‌ಪುಟ್ y(t) ಅದರ ಸ್ಥಿತಿ ಮತ್ತು ಈಗಿನ ಇನ್‌ಪುಟ್ ಮೇಲೆ ಅವಲಂಬಿತ.

ಸರಳ RNN ನಲ್ಲಿ y(t) = h(t) (ಒಂದೇ ಸಂಖ್ಯೆ). LSTM/GRU ನಲ್ಲಿ ಇವೆರಡೂ ಬೇರೆ ಬೇರೆ ಆಗಿರಬಹುದು (ಭಾಗ ೭).

೪. ಇನ್‌ಪುಟ್-ಔಟ್‌ಪುಟ್ ಅನುಕ್ರಮಗಳ ನಾಲ್ಕು ವಿಧಗಳು



ಚಿತ್ರ ೩: ನಾಲ್ಕು ವಿನ್ಯಾಸಗಳು — x = ಇನ್‌ಪುಟ್, y = ಔಟ್‌ಪುಟ್ (ಇಲ್ಲದ ಸ್ಥಾನ = ನಿರ್ಲಕ್ಷಿಸಿದ್ದು)

ವಿನ್ಯಾಸ	ವಿವರ	ನಿಜ ಜೀವನದ ಉದಾಹರಣೆ
Sequence to Vector (Many to One)	ಅನುಕ್ರಮ ಇನ್‌ಪುಟ್ → ಒಂದೇ ಔಟ್‌ಪುಟ್ (ಕೊನೆಯ ಹೆಜ್ಜೆಯದು ಮಾತ್ರ ಬಳಕೆ)	ಸಿನಿಮಾ ವಿಮರ್ಶೆ → positive/negative; ಧ್ವನಿ → ಭಾಷೆ ಗುರುತಿಸುವಿಕೆ
Sequence to Sequence (Many to Many)	ಅನುಕ್ರಮ ಇನ್‌ಪುಟ್ → ಅನುಕ್ರಮ ಔಟ್‌ಪುಟ್ (ಪ್ರತಿ ಹೆಜ್ಜೆಗೂ ಒಂದು)	**ಷೇರು ಬೆಲೆ:** ಕಳೆದ N ದಿನಗಳ ಬೆಲೆ ಕೊಡಿ; ನೆಟ್‌ವರ್ಕ್ ಒಂದು ದಿನ ಮುಂದಿನ ಬೆಲೆ ಕೊಡಬೇಕು (N−1 ದಿನಗಳ ಹಿಂದಿನಿಂದ ನಾಳೆಯವರೆಗೆ)
Vector to Sequence (One to Many)	ಒಂದೇ ಇನ್‌ಪುಟ್ → ಅನುಕ್ರಮ ಔಟ್‌ಪುಟ್	ಒಂದು ಚಿತ್ರ → ಅದರ ಬಗ್ಗೆ ವಾಕ್ಯ (ಚಿತ್ರ ಶೀರ್ಷಿಕೆ); ಒಂದು ಸಂಗೀತ ಶೈಲಿ → ರಾಗ
Encoder–Decoder	ಮೊದಲು ಇನ್‌ಪುಟ್ ಅನುಕ್ರಮವನ್ನು ಒಂದು ಸಾರಾಂಶ ಸದಿಶಕ್ಕೆ (encoder), ನಂತರ ಅದರಿಂದ ಔಟ್‌ಪುಟ್ ಅನುಕ್ರಮ (decoder)	ಕನ್ನಡ ವಾಕ್ಯ → English ವಾಕ್ಯ (ಯಂತ್ರ ಅನುವಾದ) — ಇನ್‌ಪುಟ್ ಮತ್ತು ಔಟ್‌ಪುಟ್ ಉದ್ದ ಬೇರೆ ಆಗಿರಬಹುದು

> **Keras ನಲ್ಲಿ**
> ಪ್ರತಿ ಹೆಜ್ಜೆಗೂ ಔಟ್‌ಪುಟ್ ಬೇಕಾದರೆ (many-to-many ಅಥವಾ ಇನ್ನೊಂದು ಆವರ್ತಕ ಪದರಕ್ಕೆ ಕೊಡಲು) `return_sequences=True` ; ಕೇವಲ ಕೊನೆಯ ಔಟ್‌ಪುಟ್ ಬೇಕಾದರೆ (many-to-one) ಡೀಫಾಲ್ಟ್ `False` . ಭಾಗ ೯ ರಲ್ಲಿ ಇದನ್ನು ಬಳಸಿದ್ದೇವೆ.

೫. RNN ತರಬೇತಿ (BPTT) ಮತ್ತು Keras SimpleRNN

Backpropagation Through Time (BPTT)

RNN ಅನ್ನು ತರಬೇತಿ ಮಾಡಲು ನೆಟ್‌ವರ್ಕ್ ಅನ್ನು **ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿ** (ಚಿತ್ರ ೨ ರಂತೆ ಹಲವು ಪ್ರತಿಗಳು, ಒಂದೇ ತೂಕಗಳೊಂದಿಗೆ) ಸಾಮಾನ್ಯ backpropagation ಅನ್ವಯಿಸುತ್ತೇವೆ. ಇದಕ್ಕೆ **Backpropagation Through Time (BPTT)** ಎನ್ನುತ್ತಾರೆ. ಔಟ್‌ಪುಟ್ ಅನುಕ್ರಮವನ್ನು ವೆಚ್ಚ ಫಂಕ್ಷನ್ C ಯಿಂದ ಮೌಲ್ಯಮಾಪನ ಮಾಡುತ್ತೇವೆ: `C( Y(t_min), Y(t_min+1), …, Y(t_max) )` — ಇಲ್ಲಿ t_min, t_max = ಮೊದಲ ಮತ್ತು ಕೊನೆಯ ಔಟ್‌ಪುಟ್ ಹೆಜ್ಜೆಗಳು. ಗ್ರೇಡಿಯಂಟ್‌ಗಳು ಎಲ್ಲ ಹೆಜ್ಜೆಗಳಿಂದ ಹಿಂದಕ್ಕೆ ಹರಿದು ಒಂದೇ ತೂಕಗಳಿಗೆ ಕೂಡುತ್ತವೆ.

> **ಹೋಲಿಕೆ: ಕ್ರಿಕೆಟ್ ಪಂದ್ಯದ ವಿಮರ್ಶೆ**
> ಪಂದ್ಯ ಸೋತ ಮೇಲೆ ಕೋಚ್ ಕೊನೆಯ ಓವರ್‌ನಿಂದ ಹಿಂದಕ್ಕೆ ಒಂದೊಂದು ಓವರ್ ಪರಿಶೀಲಿಸಿ “ಯಾವ ಹಂತದಲ್ಲಿ ಯಾವ ನಿರ್ಧಾರ ತಪ್ಪಾಯಿತು” ಎಂದು ಹುಡುಕುವಂತೆ — BPTT ಕಾಲದಲ್ಲಿ ಹಿಂದಕ್ಕೆ ಹೋಗಿ ತಪ್ಪಿನ ಹೊಣೆ ಹಂಚುತ್ತದೆ.

Keras: SimpleRNN

`SimpleRNN` = “ಪೂರ್ಣ-ಸಂಪರ್ಕ RNN, ಇದರ ಔಟ್‌ಪುಟ್ ಹೊಸ ಇನ್‌ಪುಟ್‌ಗೆ ಮರಳಿ ಸೇರುತ್ತದೆ”. ಮುಖ್ಯ ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳು (ಸ್ಲೈಡ್ 11): `units` (ಔಟ್‌ಪುಟ್ ಆಯಾಮ), `activation` (ಡೀಫಾಲ್ಟ್ `"tanh"`), `use_bias=True` , `kernel_initializer="glorot_uniform"` (ಇನ್‌ಪುಟ್ ತೂಕ), `recurrent_initializer="orthogonal"` (ಹಿಂದಿನ ಸ್ಥಿತಿಯ ತೂಕ), `dropout` , `recurrent_dropout` , `return_sequences=False` .

ಸ್ಲೈಡ್‌ನ ಮಾದರಿ ಕೋಡ್ (ಪಠ್ಯದ ಮುಂದಿನ ಅಕ್ಷರ ಊಹಿಸುವ ಮಾದರಿ; ಕೋಡ್ ಭಾಗಶಃ ಮಾತ್ರ ಕಾಣುತ್ತದೆ):

```
model = Sequential()
model.add(Input((input_chars, num_characters)))        # input_chars ಅಕ್ಷರಗಳ ಅನುಕ್ರಮ, ಪ್ರತಿ ಅಕ್ಷರ one-hot (num_characters ಉದ್ದ)
model.add(SimpleRNN(128, activation='tanh'))           # 128 ಆವರ್ತಕ ನ್ಯೂರಾನ್‌ಗಳು
model.add(Dense(num_characters, activation='softmax')) # ಮುಂದಿನ ಅಕ್ಷರ ಯಾವುದು? — ಪ್ರತಿ ಅಕ್ಷರಕ್ಕೆ ಸಂಭವನೀಯತೆ
model.summary()

model.compile(loss='categorical_crossentropy', optimizer='adam')
history = model.fit(X, Y, batch_size=32, epochs=100)
```

ಲೆಕ್ಕ: ಈ ಮಾದರಿಯ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ನನ್ನ ಉದಾಹರಣೆ: num_characters = 40)

```
SimpleRNN(128): 128 × (40 + 128) + 128 = 128 × 168 + 128 = 21,632
   (ಇನ್‌ಪುಟ್ ತೂಕ 40×128 = 5,120; ಪುನರಾವರ್ತಕ ತೂಕ 128×128 = 16,384; bias 128)
Dense(40)      : 128 × 40 + 40 = 5,160
ಒಟ್ಟು           = 26,792
```

ಹೋಲಿಕೆ: ಪುನರಾವರ್ತಕ ತೂಕಗಳೇ (16,384) ಪ್ಯಾರಾಮೀಟರ್‌ಗಳ ಬಹುಪಾಲು — ಅವು "ನೆನಪಿನ" ತೂಕಗಳು.

೬. ಸರಳ RNN ನ ದೌರ್ಬಲ್ಯ: ಮಾಯವಾಗುವ / ಸ್ಫೋಟಿಸುವ ಗ್ರೇಡಿಯಂಟ್

ಉದ್ದದ ಅನುಕ್ರಮದಲ್ಲಿ (ಅನೇಕ ಕಾಲ ಹೆಜ್ಜೆಗಳು) ಬಿಚ್ಚಿದ ನೆಟ್‌ವರ್ಕ್ **ತುಂಬ ಆಳ** ವಾಗುತ್ತದೆ. ಎಲ್ಲ ಆಳ ನೆಟ್‌ವರ್ಕ್‌ಗಳಂತೆ **vanishing ಮತ್ತು exploding gradients** ಸಮಸ್ಯೆ ಬರುತ್ತದೆ. ಫಲಿತಾಂಶ: ದೀರ್ಘ ತರಬೇತಿಯಲ್ಲಿ RNN **ಆರಂಭದ ಇನ್‌ಪುಟ್‌ಗಳನ್ನು ಕ್ರಮೇಣ ಮರೆಯುತ್ತದೆ**.

ಲೆಕ್ಕ: ಗ್ರೇಡಿಯಂಟ್ ಏಕೆ ಮಾಯವಾಗುತ್ತದೆ ಅಥವಾ ಸ್ಫೋಟಿಸುತ್ತದೆ? (ಚಿತ್ರಣಾತ್ಮಕ)

BPTT ಯಲ್ಲಿ ಪ್ರತಿ ಹೆಜ್ಜೆಯ ಹಿಂದಕ್ಕೆ ಹೋಗುವಾಗ ಗ್ರೇಡಿಯಂಟ್ ಒಂದು ಗುಣಕದಿಂದ ಗುಣಿತವಾಗುತ್ತದೆ ಎಂದುಕೊಳ್ಳಿ (ಇದು ಮೂಲತಃ wy ಮತ್ತು tanh ವ್ಯುತ್ಪನ್ನದ ಗುಣಲಬ್ಧ):

ಹೆಜ್ಜೆಗಳು T	ಗುಣಕ 0.9 (ಕುಗ್ಗುವ): 0.9^T	ಗುಣಕ 1.1 (ಹಿಗ್ಗುವ): 1.1^T
10	0.349	2.59
50	0.00515	117
100	0.0000266 (ಬಹುತೇಕ 0)	13,781 (ಸ್ಫೋಟ!)

100 ಹೆಜ್ಜೆ ಹಿಂದಿನ ಇನ್‌ಪುಟ್‌ಗೆ ಬರುವ ತಿದ್ದುಪಡಿ ಸಂದೇಶ ಒಂದೋ ಬಹುತೇಕ ಸೊನ್ನೆ (ಕಲಿಯಲಾಗದು) ಅಥವಾ ಅಗಾಧ (ಅಸ್ಥಿರ, ತೂಕಗಳು NaN ಆಗಬಹುದು).

ಹೋಲಿಕೆ: ಕಿವಿಯಿಂದ ಕಿವಿಗೆ ಸಂದೇಶ

ನೂರು ಜನರ ಸಾಲಿನಲ್ಲಿ ಪ್ರತಿಯೊಬ್ಬರೂ ಸಂದೇಶದ 90% ಮಾತ್ರ ಹೇಳಿದರೆ ಕೊನೆಯವನಿಗೆ ಏನೂ ಉಳಿಯುವುದಿಲ್ಲ. ವಿಮರ್ಶೆಯ ಆರಂಭದಲ್ಲಿ "ಅದ್ಭುತ ಸಿನಿಮಾ!" ಎಂದು ಬರೆದು ಕೊನೆಗೆ ಮೂವತ್ತು ವಾಕ್ಯ ನಂತರ ತೀರ್ಪು ಕೊಡಬೇಕಾದರೆ ಸರಳ RNN ಆರಂಭದ ಪದಗಳನ್ನು ಮರೆತಿರುತ್ತದೆ.

ಸಾಂಪ್ರದಾಯಿಕ ಪರಿಹಾರಗಳು

- ಒಳ್ಳೆಯ ಪ್ಯಾರಾಮೀಟರ್ ಆರಂಭೀಕರಣ (initialization) — ಉದಾ: `recurrent_initializer="orthogonal"`
- ಸೂಕ್ತ ಸಕ್ರಿಯಕರಣ ಫಂಕ್ಷನ್ ಆಯ್ಕೆ
- Normalization / regularization ಬಳಕೆ

- ಸೂಕ್ತ optimizer ಆಯ್ಕೆ (ಹಿಂದಿನ ಉಪನ್ಯಾಸ ನೋಡಿ)
- **Truncated BPTT:** ತರಬೇತಿಯಲ್ಲಿ ಹಿಂದಕ್ಕೆ ಹೋಗುವ ಕಾಲ ಹೆಜ್ಜೆಗಳ ಸಂಖ್ಯೆಯನ್ನು ಮಿತಗೊಳಿಸುವುದು

ಆದರೆ ಮೂಲ ಪರಿಹಾರ ಹೊಸ ರೀತಿಯ ಕೋಶ: **Long Short-Term Memory (LSTM)**.

2. LSTM — ಗೇಟ್‌ಗಳ ಮೂಲಕ ದೀರ್ಘ ನೆನಪು

LSTM (Jürgen Schmidhuber, Sepp Hochreiter, 1997) ಸರಳ RNN ಗೆ ಒಂದು ಹೆಚ್ಚುವರಿ "ದೀರ್ಘ ನೆನಪಿನ ಹೆದ್ದಾರಿ" — **ಕೋಶ ಸ್ಥಿತಿ C(t)** — ಮತ್ತು ಮಾಹಿತಿ ಹರಿವನ್ನು ನಿಯಂತ್ರಿಸುವ **ಮೂರು ಗೇಟ್‌ಗಳನ್ನು** ಸೇರಿಸುತ್ತದೆ.

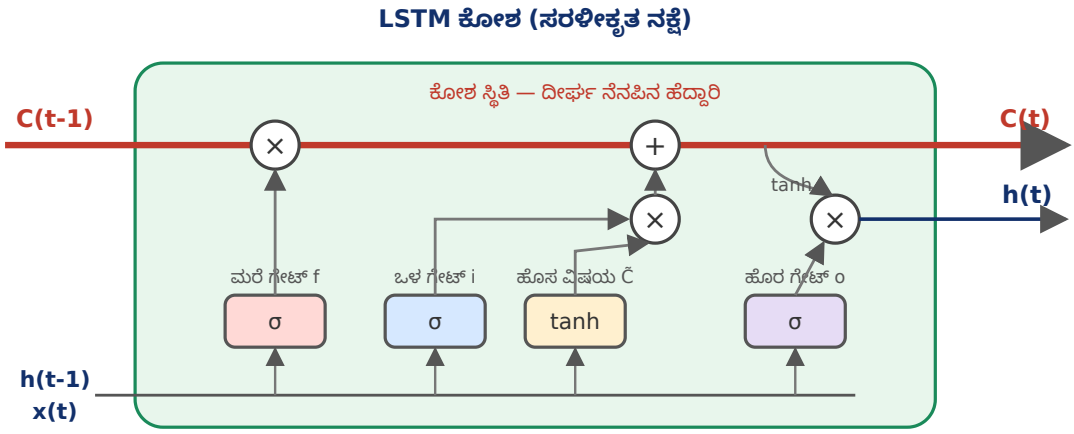
ಗೇಟ್ ಎಂದರೇನು? (ಸ್ಲೈಡ್ 13)

ಗೇಟ್ ಒಂದು **sigmoid** ಫಂಕ್ಷನ್ ಔಟ್‌ಪುಟ್ (0 ರಿಂದ 1) — ಅದನ್ನು ಸದಿಶದ ಪ್ರತಿ ಅಂಶದ ಜೊತೆ ಗುಣಿಸಲಾಗುತ್ತದೆ. **0 = ಬಾಗಿಲು ಮುಚ್ಚಿದೆ (ಏನೂ ಹಾದುಹೋಗದು), 1 = ತೆರೆದಿದೆ (ಎಲ್ಲವೂ ಹಾದುಹೋಗುತ್ತದೆ), ನಡುವೆ = ಭಾಗಶಃ ತೆರೆದಿದೆ.**

> **ಸ್ಲೈಡ್‌ನ ಉದಾಹರಣೆ: ಸದಿಶ X ಗೇಟ್ ಮೂಲಕ ಹಾದಾಗ (ಮೊದಲ 5 ಅಂಶಗಳು)**
> X = [−0.222, −0.857, 0.934, −0.577, 0.127, ...]

ಗೇಟ್ ಮೌಲ್ಯ	ಫಲಿತಾಂಶ (X × ಗೇಟ್)	ಅರ್ಥ
0	[0, 0, 0, 0, 0, ...]	ಸಂಪೂರ್ಣ ಮುಚ್ಚಿದೆ
1	[−0.222, −0.857, 0.934, −0.577, 0.127, ...]	ಸಂಪೂರ್ಣ ತೆರೆದಿದೆ
0.3	[−0.067, −0.257, 0.280, −0.173, 0.038, ...]	30% ಮಾತ್ರ ಹಾದುಹೋಗುತ್ತದೆ
0.9	[−0.200, −0.771, 0.840, −0.519, 0.114, ...]	90% ಹಾದುಹೋಗುತ್ತದೆ

> **ಹೋಲಿಕೆ: ನೀರಿನ ಕವಾಟ ಮತ್ತು ದಿನಚರಿ ಪುಸ್ತಕ**
> ಗೇಟ್ = ನೀರಿನ ಕವಾಟ (ಎಷ್ಟು ನೀರು ಹರಿಯಬೇಕು ಎಂದು ನಿಯಂತ್ರಿಸುತ್ತದೆ). ಕೋಶ ಸ್ಥಿತಿ = ನಿಮ್ಮ ದಿನಚರಿ ಪುಸ್ತಕ: ಪ್ರತಿ ದಿನ (ಹೆಜ್ಜೆ) ಹಳೆಯ ಅನಗತ್ಯ ಟಿಪ್ಪಣಿಗಳನ್ನು ಅಳಿಸುತ್ತೀರಿ (**forget gate**), ಇಂದಿನ ಮುಖ್ಯ ಸಂಗತಿಗಳನ್ನು ಬರೆಯುತ್ತೀರಿ (**input gate**), ಮತ್ತು ಯಾರಿಗಾದರೂ ಹೇಳುವಾಗ ಪುಸ್ತಕದ ಯಾವ ಭಾಗವನ್ನು ಹೇಳಬೇಕು ಎಂದು ಆರಿಸುತ್ತೀರಿ (**output gate**).



ಚಿತ್ರ ೪: LSTM ಕೋಶ — ಕೆಂಪು ಹೆದ್ದಾರಿ ಕೋಶ ಸ್ಥಿತಿ C; ಮೂರು σ ಗೇಟ್‌ಗಳು ಮತ್ತು ಒಂದು tanh ಪದರ

ಮೂರು ಗೇಟ್‌ಗಳು ಮತ್ತು ಸೂತ್ರಗಳು

ಎಲ್ಲವೂ ಹಿಂದಿನ ಸ್ಥಿತಿ $h(t-1)$ ಮತ್ತು ಈಗಿನ ಇನ್‌ಪುಟ್ $x(t)$ ಯ ಜೋಡಣೆ $[h(t-1), x(t)]$ ಅನ್ನು ಬಳಸುತ್ತವೆ (σ = sigmoid, $*$ = ಅಂಶ-ವಾರು ಗುಣಾಕಾರ):

ಹಂತ	ಸೂತ್ರ	ಕೆಲಸ
೧. ಮರೆ ಗೇಟ್ (Forget)	$f(t) = \sigma(W_f \cdot [h(t-1), x(t)] + b_f)$	ಹಳೆಯ ಕೋಶ ಸ್ಥಿತಿಯ ಯಾವ ಭಾಗ ಉಳಿಸಬೇಕು/ಅಳಿಸಬೇಕು
೨. ಒಳ ಗೇಟ್ (Input)	$i(t) = \sigma(W_i \cdot [h(t-1), x(t)] + b_i)$ $\tilde{C}(t) = \tanh(W_C \cdot [h(t-1), x(t)] + b_C)$	ಯಾವ ಹೊಸ ಮಾಹಿತಿ ($\tilde{C}$) ಎಷ್ಟು ಸೇರಿಸಬೇಕು
೩. ಕೋಶ ಸ್ಥಿತಿ ನವೀಕರಣ	$C(t) = f(t) * C(t-1) + i(t) * \tilde{C}(t)$	ಹಳೆಯದನ್ನು ಕತ್ತರಿಸಿ + ಹೊಸದನ್ನು ಸೇರಿಸಿ
೪. ಹೊರ ಗೇಟ್ (Output)	$o(t) = \sigma(W_o \cdot [h(t-1), x(t)] + b_o)$ $h(t) = o(t) * \tanh(C(t))$	ಕೋಶ ಸ್ಥಿತಿಯ ಯಾವ ಭಾಗವನ್ನು ಹೊರಗೆ (ಮುಂದಿನ ಘಟಕಕ್ಕೆ) ಕಳುಹಿಸಬೇಕು

ಹಂತ-ಹಂತದ ಲೆಕ್ಕ (ನನ್ನ ಸರಳ ಉದಾಹರಣೆ: ಒಂದೇ ಸಂಖ್ಯೆಯ ಕೋಶ)

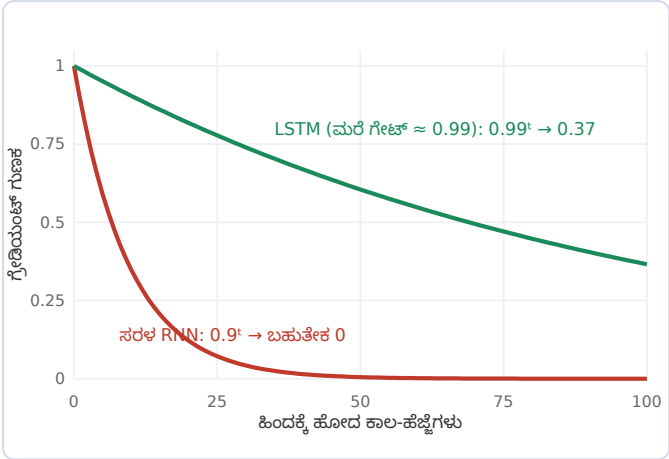
ಹಿಂದಿನ ಕೋಶ ಸ್ಥಿತಿ $C(t-1) = 2.0$. ಗೇಟ್‌ಗಳು ಲೆಕ್ಕ ಹಾಕಿದ ಮೌಲ್ಯಗಳು: $f = 0.9$, $i = 0.5$, ಹೊಸ ಅಭ್ಯರ್ಥಿ $\tilde{C} = 0.8$, $o = 0.7$.

$$C(t) = f \times C(t-1) + i \times \tilde{C} = 0.9 \times 2.0 + 0.5 \times 0.8 = 1.8 + 0.4 = 2.2$$
$$h(t) = o \times \tanh(C(t)) = 0.7 \times \tanh(2.2) = 0.7 \times 0.9757 = 0.683$$

ಎಲ್ಲವನ್ನೂ ಮರೆತರೆ: $f = 0$ ಆದರೆ $C(t) = 0 + 0.5 \times 0.8 =$ **0.4** (ಹಳೆಯ ನೆನಪು 2.0 ಸಂಪೂರ್ಣ ಅಳಿಸಿತು). **ಎಲ್ಲವನ್ನೂ ಉಳಿಸಿದರೆ:** $f = 1$, $i = 0$ ಆದರೆ $C(t) = C(t-1) = 2.0$ (ನೆನಪು ಹಾಗೆಯೇ ಮುಂದಕ್ಕೆ ಹರಿಯುತ್ತದೆ).

LSTM ಏಕೆ ಮರೆಯುವುದಿಲ್ಲ? (ಗ್ರೇಡಿಯಂಟ್ ದೃಷ್ಟಿಯಿಂದ)

ಕೋಶ ಸ್ಥಿತಿಯ ಮೇಲೆ ಹರಿಯುವ ಗ್ರೇಡಿಯಂಟ್ ಪ್ರತಿ ಹೆಜ್ಜೆಯಲ್ಲಿ ಕೇವಲ **f(t) ಯಿಂದ** ಗುಣಿತವಾಗುತ್ತದೆ (ಸರಳ RNN ನಂತೆ ಪ್ರತಿ ಬಾರಿ ತೂಕ ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಮತ್ತು tanh ವ್ಯುತ್ಪನ್ನದಿಂದಲ್ಲ). ಮರೆ ಗೇಟ್ 1 ಕ್ಕೆ ಹತ್ತಿರವಿದ್ದರೆ ಗ್ರೇಡಿಯಂಟ್ ಬಹುತೇಕ ಕುಗ್ಗುವುದಿಲ್ಲ.



ಚಿತ್ರ ೫: 100 ಹೆಜ್ಜೆ ಹಿಂದಕ್ಕೆ ಗ್ರೇಡಿಯಂಟ್ ಹೇಗೆ ಉಳಿಯುತ್ತದೆ (ಚಿತ್ರಣ)

ಲೆಕ್ಕ

```
ಸರಳ RNN (ಗುಣಕ 0.9): 0.9¹⁰⁰ = 0.0000266
LSTM (f = 0.99)    : 0.99¹⁰⁰ = 0.366
```

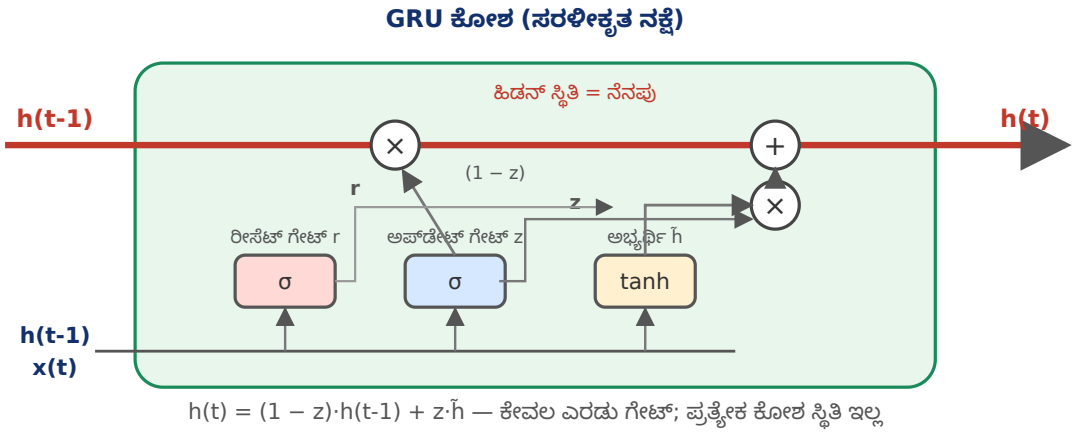
ಅಂದರೆ 100 ಹೆಜ್ಜೆ ಹಿಂದಿನ ಮಾಹಿತಿಗೆ LSTM ನಲ್ಲಿ ಸುಮಾರು 37% ಸಂಕೇತ ತಲುಪುತ್ತದೆ; ಸರಳ RNN ನಲ್ಲಿ ಕೇವಲ ಸುಮಾರು 0.003%. (ಇದು ಕೇವಲ ಚಿತ್ರಣಾತ್ಮಕ; ನಿಜವಾದ ಮೌಲ್ಯಗಳು ಸಂದರ್ಭಕ್ಕೆ ತಕ್ಕಂತೆ ಬದಲಾಗುತ್ತವೆ.)

Keras LSTM ಪದರದ ಮುಖ್ಯ ಆರ್ಗ್ಯುಮೆಂಟ್‌ಗಳು (ಸ್ಲೈಡ್ 19)

ಆರ್ಗ್ಯುಮೆಂಟ್	ಡೀಫಾಲ್ಟ್	ಅರ್ಥ
`units`	—	ಔಟ್‌ಪುಟ್ (hidden) ಆಯಾಮ
`activation`	"tanh"	ಹೊಸ ವಿಷಯ C̃ ಮತ್ತು ಔಟ್‌ಪುಟ್‌ನ tanh
`recurrent_activation`	"sigmoid"	ಗೇಟ್‌ಗಳ ಸಕ್ರಿಯಕರಣ
`unit_forget_bias`	True	ಮರೆ ಗೇಟ್ bias ಅನ್ನು ಆರಂಭದಲ್ಲಿ 1 ಮಾಡಿ ನೆನಪು ಉಳಿಸಲು ಪ್ರೇರೇಪಿಸುತ್ತದೆ
`return_sequences`	False	ಪ್ರತಿ ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್ ಹಿಂದಿರುಗಿಸಬೇಕೇ
`dropout`, `recurrent_dropout`	0.0	ಇನ್‌ಪುಟ್ ಮತ್ತು ಪುನರಾವರ್ತಕ ಸಂಪರ್ಕದ dropout (overfitting ತಡೆಗೆ)

೮. GRU — ಸರಳೀಕೃತ ಗೇಟ್ ಕೋಶ ಮತ್ತು LSTM ಹೋಲಿಕೆ

Gated Recurrent Unit (GRU) (Cho et al., 2014) LSTM ನ ಸರಳ ರೂಪ: **ಎರಡು ಗೇಟ್‌ಗಳು** (reset ಮತ್ತು update) ಮತ್ತು ಪ್ರತ್ಯೇಕ ಕೋಶ ಸ್ಥಿತಿ ಇಲ್ಲ — ಹಿಡನ್ ಸ್ಥಿತಿ h(t) ಅದೇ ನೆನಪು. LSTM ಮತ್ತು GRU ಎರಡರಲ್ಲೂ ನೆನಪಿನ ಕೋಶ ಇರುವುದರಿಂದ ಗ್ರೇಡಿಯಂಟ್ ಒಂದು ಮಟ್ಟಕ್ಕಿಂತ ಕೆಳಗೆ ಇಳಿಯುವುದಿಲ್ಲ.



ಚಿತ್ರ ೬: GRU ಕೋಶ

ಗೇಟ್	ಸೂತ್ರ	ಕೆಲಸ
Reset ಗೇಟ್ r(t)	r(t) = σ(W_r x(t) + U_r h(t−1))	ಹೊಸ ಅಭ್ಯರ್ಥಿಯನ್ನು ಲೆಕ್ಕ ಹಾಕುವಾಗ ಹಳೆಯ ಸ್ಥಿತಿಯ ಎಷ್ಟನ್ನು ಬಳಸಬೇಕು
ಅಭ್ಯರ್ಥಿ ಸ್ಥಿತಿ h̃(t)	h̃(t) = tanh(W x(t) + U (r(t) ⊙ h(t−1)))	ಹೊಸ ಸಂಭಾವ್ಯ ವಿಷಯ
Update ಗೇಟ್ z(t)	z(t) = σ(W_z x(t) + U_z h(t−1))	ಹಳೆಯ ಸ್ಥಿತಿ ಉಳಿಸಬೇಕೇ ಅಥವಾ ಹೊಸದರಿಂದ ಬದಲಿಸಬೇಕೇ
ಹೊಸ ಸ್ಥಿತಿ	h(t) = (1 − z(t)) ⊙ h(t−1) + z(t) ⊙ h̃(t)	ಹಳೆಯ ಮತ್ತು ಹೊಸದರ ತೂಕ ಮಿಶ್ರಣ

ಲೆಕ್ಕ: Update ಗೇಟ್ ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ? (ನನ್ನ ಉದಾಹರಣೆ)

h(t−1) = 0.5, ಅಭ್ಯರ್ಥಿ h̃ = 0.9:

```
z = 0.2 :  h(t) = 0.8 × 0.5 + 0.2 × 0.9 = 0.40 + 0.18 = 0.58   (ಹೆಚ್ಚಾಗಿ ಹಳೆಯದನ್ನು ಉಳಿಸಿದೆ)
z = 0   :  h(t) = 1.0 × 0.5 + 0 × 0.9   = 0.50                 (ಹಳೆಯದು ಹಾಗೆಯೇ — ಸಂಪೂರ್ಣ ನೆನಪು)
z = 1   :  h(t) = 0 × 0.5 + 1.0 × 0.9   = 0.90                 (ಹಳೆಯದು ಸಂಪೂರ್ಣ ಬದಲಿ)
```

LSTM ನ ಮರೆ ಗೇಟ್ f ಮತ್ತು ಒಳ ಗೇಟ್ i ಅನ್ನು ಇಲ್ಲಿ ಒಂದೇ z ಮತ್ತು (1 − z) ಆಗಿ ಜೋಡಿಸಲಾಗಿದೆ.

LSTM vs GRU

ಅಂಶ	LSTM	GRU
ಗೇಟ್‌ಗಳು	3 (forget, input, output)	2 (reset, update)
ಸ್ಥಿತಿಗಳು	ಕೋಶ ಸ್ಥಿತಿ C(t) + ಹಿಡನ್ h(t)	ಕೇವಲ ಹಿಡನ್ h(t)
ತೂಕ ಬ್ಲಾಕ್‌ಗಳು	4	3
ಪ್ಯಾರಾಮೀಟರ್ ಸೂತ್ರ (Keras)	4 × (units × (input + units) + units)	3 × (units × (input + units) + 2 × units) (ಡೀಫಾಲ್ಟ್ reset_after=True ಗೆ)
ವೇಗ	ಹೆಚ್ಚು ಲೆಕ್ಕಾಚಾರ	ಕಡಿಮೆ ಪ್ಯಾರಾಮೀಟರ್ → ಸಾಮಾನ್ಯವಾಗಿ ವೇಗ

೯. IMDB ವಿಮರ್ಶೆಗಳ ಸೆಂಟಿಮೆಂಟ್: LSTM ಮಾದರಿ

ಹಿಂದಿನ ಉಪನ್ಯಾಸಗಳ ಅದೇ IMDB ಸಮಸ್ಯೆ (ವಿಮರ್ಶೆ positive ಯೋ negative ಯೋ) — ಈ ಬಾರಿ LSTM ನಿಂದ. ಇದು **Many-to-One** ಕೆಲಸ: ಇಡೀ ವಿಮರ್ಶೆ (200 ಪದಗಳ ಅನುಕ್ರಮ) ಓದಿ ಒಂದೇ ತೀರ್ಪು.

ಹಂತ ೧: ಲೈಬ್ರರಿ ಮತ್ತು ದತ್ತಾಂಶ

```
import numpy as np
import matplotlib.pyplot as plt
import time
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, LSTM, Embedding
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.optimizers import Adam

max_words = 10000       # ತರಬೇತಿಯಲ್ಲಿ ಬಳಸುವ ಗರಿಷ್ಠ ಪದಗಳ ಸಂಖ್ಯೆ (ಅತಿ ಹೆಚ್ಚು ಬಳಕೆಯ 10,000)
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_words)
```

ಹಂತ ೨: ವಿಮರ್ಶೆಗಳನ್ನು 200 ಪದಗಳಿಗೆ ಸಮಗೊಳಿಸುವುದು

```
maxlen = 200   # ಎಲ್ಲ ವಿಮರ್ಶೆಗಳ ಉದ್ದ ಸಮಾನ: ಉದ್ದವಾದರೆ ಹೆಚ್ಚುವರಿ ಪದ ತೆಗೆಯುತ್ತದೆ, ಚಿಕ್ಕದಾದರೆ 0 (padding) ಸೇರಿಸುತ್ತದೆ
x_train2 = pad_sequences(x_train, maxlen=maxlen)
x_test2  = pad_sequences(x_test,  maxlen=maxlen)
print('Training matrix shape:', x_train2.shape, 'Testing matrix shape:',
x_test2.shape)
# ಫಲಿತಾಂಶ: Training matrix shape: (25000, 200) Testing matrix shape: (25000, 200)
```

ಹಂತ ೩: ಮಾದರಿ

```
model = Sequential()
model.add(Input(shape=(maxlen,)))                    # (batch, 200): ಪದ ಸೂಚಿಗಳು
model.add(Embedding(max_words, 128))                 # → (batch, 200, 128): ಪ್ರತಿ ಪದಕ್ಕೆ 128
ಉದ್ದದ ಸದಿಶ
model.add(LSTM(64, return_sequences=True))           # → (batch, 200, 64): ಪ್ರತಿ ಹೆಜ್ಜೆಯ
ಔಟ್‌ಪುಟ್ (many-to-many)
model.add(LSTM(24))                                  # → (batch, 24): ಕೊನೆಯ ಹೆಜ್ಜೆ ಮಾತ್ರ
(many-to-one)
model.add(Dense(1, activation='sigmoid'))            # → (batch, 1): positive ಸಂಭವನೀಯತೆ
model.summary()

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

ಮೊದಲ LSTM ಗೆ `return_sequences=True` ಇರುವುದರಿಂದ ಅದು ಎಲ್ಲ 200 ಹೆಜ್ಜೆಗಳ ಔಟ್‌ಪುಟ್ ಅನ್ನು ಎರಡನೇ LSTM ಗೆ ಕೊಡುತ್ತದೆ (ಪದರಗಳನ್ನು ಒಂದರ ಮೇಲೊಂದು ಜೋಡಿಸಲು ಇದು ಅಗತ್ಯ). ಎರಡನೇ LSTM ಕೊನೆಯ ಹೆಜ್ಜೆಯ 24 ಸಂಖ್ಯೆಗಳನ್ನು ಮಾತ್ರ ಕೊಡುತ್ತದೆ.

ಲೆಕ್ಕ: ಆಕಾರ ಮತ್ತು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ನನ್ನ ಲೆಕ್ಕ; summary ಯ ಚಿತ್ರ ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಇಲ್ಲ)

ಪದರ	ಔಟ್‌ಪುಟ್ ಆಕಾರ	ಲೆಕ್ಕ	ಪ್ಯಾರಾ.
Embedding(10000, 128)	(200, 128)	10,000 × 128	1,280,000
LSTM(64, return_sequences)	(200, 64)	4 × (64 × (128 + 64) + 64) = 4 × 12,352	49,408
LSTM(24)	(24)	4 × (24 × (64 + 24) + 24) = 4 × 2,136	8,544
Dense(1)	(1)	24 + 1	25
ಒಟ್ಟು			**1,337,977**

ಒಳನೋಟ: ಒಟ್ಟು ಪ್ಯಾರಾಮೀಟರ್‌ಗಳಲ್ಲಿ ಸುಮಾರು **96%** (1,280,000) ಕೇವಲ Embedding ನಲ್ಲಿ; ಎರಡು LSTM ಗಳಲ್ಲಿ ಒಟ್ಟು 57,952 (≈ 4%). ಹೋಲಿಕೆ: ಹಿಂದಿನ Conv1D ಮಾದರಿ 1,387,757 ಪ್ಯಾರಾಮೀಟರ್ ಹೊಂದಿತ್ತು. ಹೆಜ್ಜೆಗಳು: 25,000 × 0.9 = 22,500; 22,500/128 = 175.8 → 176 ಹೆಜ್ಜೆ/epoch.

ಹಂತ ೪: ತರಬೇತಿ ಮತ್ತು ಮೌಲ್ಯಮಾಪನ

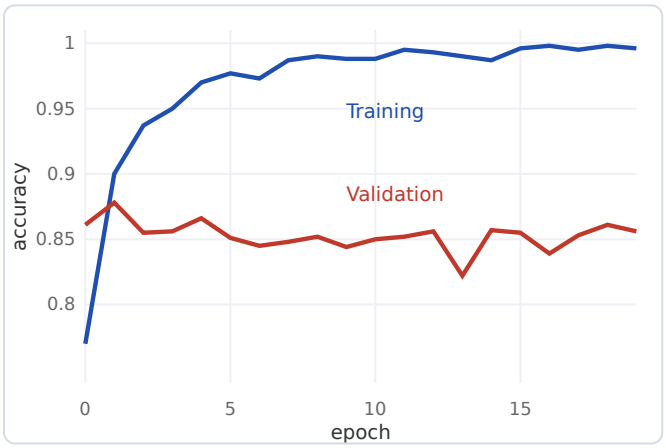
```
history = model.fit(x_train2, y_train,
                    epochs=5,
                    batch_size=128,
                    validation_split=0.1)    # 10% validation

scores = model.evaluate(x_test2, y_test, verbose=1)     # test: 25,000/32 ≈ 782 ಪೆಜ್ಜೆ
```

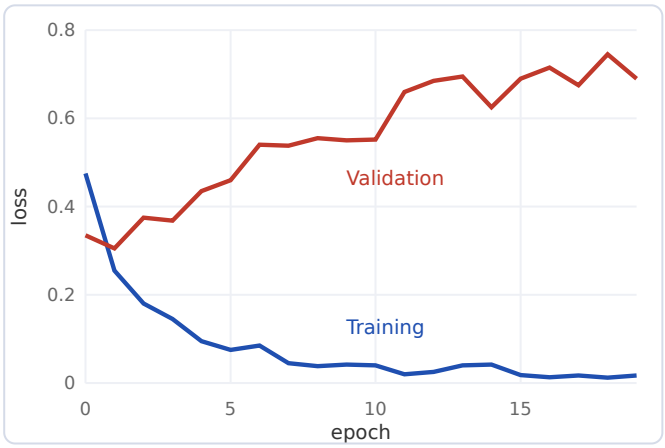
ಹಂತ ೫: ಕಲಿಕೆಯ ವಕ್ರರೇಖೆಗಳು (ಸ್ಲೈಡ್ 24)

```
plt.plot(history.history['accuracy'],     label='Training accuracy')
plt.plot(history.history['val_accuracy'], label='Validation accuracy')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.show()

plt.plot(history.history['loss'],     label='Training loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.show()
```



ಚಿತ್ರ ೭: Accuracy (ಸ್ಲೈಡ್ 24 ರ ಗ್ರಾಫ್‌ನಿಂದ ಅಂದಾಜು ಓದು)



ಚಿತ್ರ ೮: Loss (ಸ್ಲೈಡ್ 24 ರ ಗ್ರಾಫ್‌ನಿಂದ ಅಂದಾಜು ಓದು)

ಸ್ಲೈಡ್‌ನ ಗ್ರಾಫ್‌ನಿಂದ ಏನು ತಿಳಿಯುತ್ತದೆ? — ತೀವ್ರ overfitting

(ಗಮನಿಸಿ: ಸ್ಲೈಡ್ 24 ರ ಗ್ರಾಫ್‌ನಲ್ಲಿ ಸುಮಾರು 20 epochs ಇವೆ; ಸ್ಲೈಡ್ 23 ರ ಕೋಡ್ `epochs=5` ತೋರಿಸುತ್ತದೆ — ಗ್ರಾಫ್ ಹೆಚ್ಚು epochs ನ ಇನ್ನೊಂದು ಓಟದ್ದು ಆಗಿರಬಹುದು. ಮೇಲಿನ ಸಂಖ್ಯೆಗಳು ಗ್ರಾಫ್‌ನಿಂದ ಅಂದಾಜಿನಲ್ಲಿ ಓದಿದ್ದು, ನಿಖರ ಮೌಲ್ಯಗಳಲ್ಲ.)

• Training accuracy ≈ 0.77 ರಿಂದ ≈ 0.99–1.00 ಕ್ಕೆ ಏರುತ್ತದೆ, training loss ≈ 0.02 ಕ್ಕೆ ಇಳಿಯುತ್ತದೆ.

• **Validation accuracy ≈ 0.88 ರಲ್ಲಿ ಬೇಗನೆ (epoch 1–2) ಶಿಖರ ತಲುಪಿ** ನಂತರ ≈ 0.82–0.86 ನಲ್ಲಿ ಏರಿಳಿಯುತ್ತದೆ; **validation loss ≈ 0.30 ರಿಂದ ≈ 0.7 ಕ್ಕೆ ಏರುತ್ತದೆ** — ಮಾದರಿ ವಿಮರ್ಶೆಗಳನ್ನು ಉರು ಹೊಡೆಯುತ್ತಿದೆ.

ಪಾಠ: ಹೆಚ್ಚು epochs ಯಾವಾಗಲೂ ಒಳ್ಳೆಯದಲ್ಲ; early stopping (ಸುಮಾರು epoch 2) ಅಥವಾ dropout/recurrent_dropout, ಚಿಕ್ಕ ಮಾದರಿ ಬಳಸಿ (ಹಿಂದಿನ ಉಪನ್ಯಾಸಗಳು).

ಹಂತ ೬: ಹೊಸ ವಿಮರ್ಶೆಗೆ ಭವಿಷ್ಯವಾಣಿ (ಸ್ಲೈಡ್ 25)

ಸ್ಲೈಡ್‌ನಲ್ಲಿ IMDB ಯ ಒಂದು ನಿಜವಾದ ವಿಮರ್ಶೆ ತೋರಿಸಲಾಗಿದೆ — ಬರಹಗಾರ ಚಿತ್ರವನ್ನು ಟೀಕಿಸಿ "I wish I had skipped it" ಎಂದು ಬರೆದು ರೇಟಿಂಗ್ **4** ಕೊಟ್ಟಿದ್ದಾರೆ (ರೇಟಿಂಗ್ 1–4 = negative).

```
prediction = model.predict(x)     # x = ಈ ವಿಮರ್ಶೆಯನ್ನು ಸಂಖ್ಯೆಗಳಾಗಿ ಬದಲಿಸಿ, 200 ಕ್ಕೆ pad ಮಾಡಿದ್ದು
print(prediction)                 # → [[0.20414507]]
```

> **ಫಲಿತಾಂಶದ ಅರ್ಥ**
>
> ಔಟ್‌ಪುಟ್ **0.204** = "positive" ಆಗಿರುವ ಸಂಭವನೀಯತೆ ಸುಮಾರು 20%. 0.5 ಕ್ಕಿಂತ ಕಡಿಮೆ → ಮಾದರಿ ಪ್ರಕಾರ **negative** (ಸಂಭವನೀಯತೆ 80% negative). ನಿಜವಾದ ರೇಟಿಂಗ್ 4 (negative) ಜೊತೆ ಹೊಂದುತ್ತದೆ ✓.

೧೦. GRU ಮಾದರಿ ಮತ್ತು ತರಬೇತಿ ಸಮಯ ಅಳೆಯುವುದು

ಅದೇ ಮಾದರಿಯಲ್ಲಿ LSTM ಬದಲು GRU ಹಾಕಿದರೆ ಸಾಕು — ಉಳಿದ ಕೋಡ್ ಒಂದೇ:

```
from tensorflow.keras.layers import Dense, GRU, Input, Embedding

model = Sequential()
model.add(Input(shape=(maxlen,)))
model.add(Embedding(max_words, 128))
model.add(GRU(64, return_sequences=True))     # return_sequences – ಮುಂದಿನ GRU ಗೆ ಎಲ್ಲ ಹೆಜ್ಜೆಗಳ ಔಟ್‌ಪುಟ್
model.add(GRU(24))
model.add(Dense(1, activation='sigmoid'))
model.summary()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

> **ಲೆಕ್ಕ: GRU ಮಾದರಿಯ ಪ್ಯಾರಾಮೀಟರ್‌ಗಳು (ನನ್ನ ಲೆಕ್ಕ; Keras ಡೀಫಾಲ್ಟ್ reset_after=True)**
>
> ```
> GRU(64) : 3 × (64 × (128 + 64) + 2×64) = 3 × 12,416 = 37,248
> GRU(24) : 3 × (24 × (64 + 24)  + 2×24) = 3 × 2,160  =  6,480
> Embedding 1,280,000 + Dense 25
> ಒಟ್ಟು     = 1,280,000 + 37,248 + 6,480 + 25 = 1,323,753
>
> ಎರಡು ಆವರ್ತಕ ಪದರಗಳು: GRU 43,728  vs  LSTM 57,952  →  GRU ≈ 25% ಕಡಿಮೆ
> ```

ತರಬೇತಿ ಸಮಯ ಅಳೆಯುವುದು (ಸ್ಲೈಡ್ 30)

```
import time

start_time = time.time()                              # ಆರಂಭ ಸಮಯ
history = model.fit(x_train2, y_train, epochs=5, batch_size=128,
validation_split=0.1)
end_time = time.time()                                # ಅಂತ್ಯ ಸಮಯ
elapsed_time = end_time - start_time
print(f"Training took {elapsed_time} seconds")
```

ಈ ಕೋಡ್ ಅನ್ನು LSTM ಮಾದರಿ ಮತ್ತು GRU ಮಾದರಿ — ಎರಡಕ್ಕೂ ಓಡಿಸಿ ಸಮಯ ಹೋಲಿಸಿ: GRU ಗೆ ಕಡಿಮೆ ಪ್ಯಾರಾಮೀಟರ್ ಇರುವುದರಿಂದ ಸಾಮಾನ್ಯವಾಗಿ ವೇಗ. (ಸ್ಲೈಡ್‌ನಲ್ಲಿ ನಿರ್ದಿಷ್ಟ ಸಮಯಗಳು ತೋರಿಸಿಲ್ಲ — ನಿಮ್ಮ ಯಂತ್ರದಲ್ಲಿ ಓಡಿಸಿ ನೋಡಿ.) ಸಣ್ಣ ದತ್ತಾಂಶದಲ್ಲಿ ಇವೆರಡರ ನಿಖರತೆ ಹೆಚ್ಚಾಗಿ ಹತ್ತಿರವಿರುತ್ತದೆ.

೧೧. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

ಒಂದು-ನೋಟ ಹೋಲಿಕೆ

ಮಾದರಿ	ಮುಖ್ಯ ಕಲ್ಪನೆ	ಬಲ	ದೌರ್ಬಲ್ಯ
Feedforward	ಪ್ರತಿ ಇನ್‌ಪುಟ್ ಸ್ವತಂತ್ರ	ಸರಳ	ನೆನಪಿಲ್ಲ
CNN	ಸ್ಥಳೀಯ ಫಿಲ್ಟರ್, ಹಂಚಿದ ತೂಕ	ಚಿತ್ರಗಳಿಗೆ	ಕ್ರಮ-ಅವಲಂಬಿತ ದೀರ್ಘ ಸಂದರ್ಭ ಕಷ್ಟ
ಸರಳ RNN	ಹಿಂದಿನ ಔಟ್‌ಪುಟ್ ಮರಳಿ ಇನ್‌ಪುಟ್	ಅನುಕ್ರಮ, ವಿವಿಧ ಉದ್ದ	ಉದ್ದ ಅನುಕ್ರಮದಲ್ಲಿ ಮರೆಯುತ್ತದೆ (vanishing gradient)
LSTM	ಕೋಶ ಸ್ಥಿತಿ + 3 ಗೇಟ್	ದೀರ್ಘ ನೆನಪು	ಹೆಚ್ಚು ಪ್ಯಾರಾಮೀಟರ್, ನಿಧಾನ
GRU	2 ಗೇಟ್, ಒಂದೇ ಸ್ಥಿತಿ	LSTM ಗಿಂತ ಸರಳ, ವೇಗ	ಅತಿ ದೀರ್ಘ ನೆನಪಿನ ಕೆಲಸಗಳಲ್ಲಿ LSTM ಒಂದೊಮ್ಮೆ ಉತ್ತಮ

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Sequence (ಅನುಕ್ರಮ)	ಕ್ರಮಕ್ಕೆ ಅರ್ಥವಿರುವ ದತ್ತಾಂಶ — ಪಠ್ಯ, ಧ್ವನಿ, ಕಾಲ-ಸರಣಿ
Time step / frame	ಅನುಕ್ರಮದ ಒಂದು ಹೆಜ್ಜೆ (ಒಂದು ಪದ, ಒಂದು ದಿನ)
Recurrent	ತನ್ನ ಹಿಂದಿನ ಔಟ್‌ಪುಟ್ ಅನ್ನು ಮತ್ತೆ ಬಳಸುವ
Hidden state h(t)	ಕೋಶದ ಆಂತರಿಕ ನೆನಪು
Memory cell	ಹೆಜ್ಜೆಗಳ ನಡುವೆ ಸ್ಥಿತಿ ಉಳಿಸುವ ನೆಟ್‌ವರ್ಕ್ ಭಾಗ
Unrolling	ಆವರ್ತಕ ನೆಟ್‌ವರ್ಕ್ ಅನ್ನು ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿ ತೋರಿಸುವುದು
BPTT	ಕಾಲದ ಮೇಲೆ ಬಿಚ್ಚಿ ಮಾಡುವ backpropagation
Truncated BPTT	ಹಿಂದಕ್ಕೆ ಹೋಗುವ ಹೆಜ್ಜೆಗಳನ್ನು ಮಿತಗೊಳಿಸುವುದು
Gate (ಗೇಟ್)	0–1 ಮೌಲ್ಯದಿಂದ ಮಾಹಿತಿ ಹರಿವನ್ನು ನಿಯಂತ್ರಿಸುವ ಕವಾಟ
Cell state C(t)	LSTM ನ ದೀರ್ಘ ನೆನಪಿನ ಹೆದ್ದಾರಿ
Forget / Input / Output gate	ಅಳಿಸು / ಸೇರಿಸು / ಹೊರಗೆ ಕಳುಹಿಸು ಗೇಟ್
Reset / Update gate (GRU)	ಹಳೆಯ ಸ್ಥಿತಿ ಬಳಕೆ / ಬದಲಿ ನಿಯಂತ್ರಣ
return_sequences	ಪ್ರತಿ ಹೆಜ್ಜೆಯ ಔಟ್‌ಪುಟ್ ಹಿಂದಿರುಗಿಸಬೇಕೇ ಎಂಬ ಆಯ್ಕೆ
Encoder–Decoder	ಇನ್‌ಪುಟ್ ಅನ್ನು ಸಾರಾಂಶ ಮಾಡಿ, ನಂತರ ಔಟ್‌ಪುಟ್ ಅನುಕ್ರಮ ಉತ್ಪಾದಿಸುವ ರಚನೆ

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. ಸರಳ RNN ನ್ಯೂರಾನ್: $w_x = 1$, $w_y = 0.5$, $b = 0$, tanh. ಇನ್‌ಪುಟ್ $[1, 1]$, $y(0) = 0$. ಎರಡೂ ಹೆಜ್ಜೆಗಳ ಔಟ್‌ಪುಟ್?
2. SimpleRNN(64) ಗೆ ಇನ್‌ಪುಟ್ ಲಕ್ಷಣ 20 ಇದ್ದರೆ ಪ್ಯಾರಾಮೀಟರ್ ಎಷ್ಟು?
3. ಷೇರು ಬೆಲೆ ಮುನ್ಸೂಚನೆ ಮತ್ತು ಸಿನಿಮಾ ವಿಮರ್ಶೆ ಭಾವನೆ — ಇವು ಯಾವ ವಿನ್ಯಾಸ (many-to-?)?
4. ಗ್ರೇಡಿಯಂಟ್ ಗುಣಕ 0.9 ಇದ್ದು 50 ಹೆಜ್ಜೆ ಹಿಂದಕ್ಕೆ ಹೋದರೆ ಉಳಿಯುವ ಪ್ರಮಾಣ?
5. LSTM ನಲ್ಲಿ $C(t-1) = 3$, $f = 0.5$, $i = 0.2$, $\tilde{C} = 1$ ಆದರೆ $C(t)$ ಎಷ್ಟು? $o = 0.5$ ಆದರೆ $h(t) = ?$ ($\tanh(1.7) \approx 0.935$)
6. GRU ನಲ್ಲಿ $h(t-1) = 1.0$, $\tilde{h} = 0$, $z = 0.25$ ಆದರೆ $h(t)$?
7. LSTM(32) ಗೆ ಇನ್‌ಪುಟ್ ಆಯಾಮ 16 ಇದ್ದರೆ ಪ್ಯಾರಾಮೀಟರ್ ಎಷ್ಟು?
8. ಎರಡು LSTM ಪದರಗಳನ್ನು ಜೋಡಿಸುವಾಗ ಮೊದಲ LSTM ಗೆ return_sequences=True ಏಕೆ ಬೇಕು?
9. ಸ್ಲೈಡ್‌ನ ಗ್ರಾಫ್‌ನಲ್ಲಿ training accuracy ಏರುತ್ತಿದ್ದು validation loss ಏರುತ್ತಿದ್ದರೆ ಇದರ ಅರ್ಥ?

ಉತ್ತರಗಳು: (೧) tanh(1) = **0.7616**; ನಂತರ tanh(1 + 0.5×0.7616) = tanh(1.3808) = **0.8810**. (೨) 64 × (20 + 64) + 64 = 64×84 + 64 = **5,440**. (೩) ಷೇರು ಬೆಲೆ: **many-to-many** (ಅನುಕ್ರಮ → ಅನುಕ್ರಮ); ವಿಮರ್ಶೆ ಭಾವನೆ: **many-to-one**. (೪) 0.9^{50} = **0.00515** (≈ 0.5%). (೫) C(t) = 0.5×3 + 0.2×1 = 1.5 + 0.2 = **1.7**; h(t) = 0.5 × tanh(1.7) = 0.5×0.935 = **0.4675**. (೬) (1 − 0.25)×1.0 + 0.25×0 = **0.75**. (೭) 4 × (32 × (16 + 32) + 32) = 4 × (1,536 + 32) = 4 × 1,568 = **6,272**. (೮) ಎರಡನೇ LSTM ಗೆ ಪ್ರತಿ ಹೆಜ್ಜೆಯ ಇನ್‌ಪುಟ್ ಅನುಕ್ರಮ ಬೇಕು; ಕೇವಲ ಕೊನೆಯ ಔಟ್‌ಪುಟ್ ಹೋದರೆ ಅನುಕ್ರಮವೇ ಇರುವುದಿಲ್ಲ. (೯) **Overfitting** — ಮಾದರಿ ತರಬೇತಿ ದತ್ತಾಂಶವನ್ನು ಉರು ಹೊಡೆಯುತ್ತಿದೆ; early stopping / dropout / ಚಿಕ್ಕ ಮಾದರಿ ಬಳಸಿ.